A

PROJECT REPORT

ON

Sound for Silence

Submitted to:

R.T.U., KOTA (RAJASTHAN)

In partial fulfillment of the requirement for the award of degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE & ENGINEERING

Under the Supervision of:

Er. Neelam Jangir

AP in CSE Deptt

Submitted by:

Abhishek Verma (22EAOCS005)

Shekhar Chaurasiya (22EAOCS047)

Sakshi Puria (22EAOCS042)

Abhishek Rathore (22EAOCS004)

ARYA INSTITUTE OF ENGINEERING, TECHNOLOGY & MANAGEMENT, JAIPUR

RAJASTHAN TECHNICAL UNIVERSITY, KOTA (RAJASTHAN)

(2025–26)

ARYA INSTITUTE OF ENGINEERING, TECHNOLOGY & MANAGEMENT, JAIPUR

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Certificate of Completion

This is to certify that the project titled **"Sound for Silence"** has been successfully completed by **Abhishek Verma, Shekhar Chaurasiya, Sakshi Puria, and Abhishek Rathore**, a student of Bachelor of Technology, 8th Semester, at **Arya Institute of Engineering, Technology & Management, Jaipur**. The project work presented in this report is a **bonafide and satisfactory account** of the work carried out under my supervision, and is hereby **recommended for submission** as part of the fulfilment of the academic requirements for the 8th semester of the B.Tech program.

Project In-charge
Er. Pawan Sen

Submitted by
Abhishek Verma (22EAOCS005)
Shekhar Chaurasiya (22EAOCS047)
Sakshi Puria (22EAOCS042)
Abhishek Rathore (22EAOCS004)

DECLARATION BY THE CANDIDATE

We, **Abhishek Verma, Shekhar Chaurasiya, Sakshi Puria, and Abhishek Rathore** hereby declare that the project work titled **Sound for Silence** is an original and authentic piece of work carried out by us at **Arya Institute of Engineering, Technology & Management, Jaipur** in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology (B.Tech)**.

We further affirm that this work has not been submitted, either fully or partially, for the award of any other degree or diploma at any institution.

Abhishek Verma (22EAOCS005)

Shekhar Chaurasiya (22EAOCS047)

Sakshi Puria (22EAOCS042)

Abhishek Rathore (22EAOCS004)

ACKNOWLEDGEMENT

It gives us immense pleasure to express my heartfelt gratitude to all those who have supported and guided me throughout the successful completion of this final year project. We would first like to extend my sincere thanks to Dr. **Arvind Agarwal**, President, and **Dr. Puja Agarwal**, Group Director of Arya Institute of Engineering Technology and Management, Jaipur, for providing an excellent academic environment and all the necessary facilities that foster learning and innovation.

We would like to express my special thanks to **Er. Neelam Jangid**, Assistant Professor, **Computer Science and Engineering**, for his visionary leadership, valuable suggestions, and motivation throughout the course of this project.

We am also thankful to my project In-charge, **Er. Pawan Sen**, whose timely feedback, guidance, and coordination played a vital role in ensuring the quality and completion of this project work.

Last but not least, We sincerely thank all the faculty members, lab assistants, and my peers who contributed directly or indirectly with their support and inspiration during the entire project duration. This project has been a significant learning experience and a stepping stone in my professional development.

Abhishek Verma (22EAOCS005)
Shekhar Chaurasiya (22EAOCS047)
Sakshi Puria (22EAOCS042)
Abhishek Rathore (22EAOCS004)

B.Tech 8th Semester, CSE

TABLE OF CONTENTS

PRELIMINARIES

• Certificate of Completion i
• Declaration by the Candidate ii
• Acknowledgement iii
• Table of Contents iv
• List of Figures vi
• List of Tables vii

1. Introduction 1

1.1. Abstract of Project 1

1.2. Problem Specification / Need of Project 2

2. Feasibility Study 4

2.1. Technical Feasibility 4

2.2. Operational Feasibility 4

2.3. Economic Feasibility 5

3. Software Requirement Specifications 6

3.1. Introduction 6

3.2. Selection of Technology / Specific Requirements 7

4. Design 9

4.1. ER Diagram 9

4.2. Data Flow Diagram (Level 0 & Level 1) 10

4.3. Modules 12

4.4. Database 13

4.5. Input–Output Form / Screen Layout 16

5. Implementation / Technological Environment 17

5.1. Firebase Project Setup 17

5.2. Firebase Authentication Implementation 17

5.3. Cloud Firestore — Data Seeding 20

5.4. Firestore Security Rules Implementation 21

5.5. Firebase Hosting — Deployment 23

5.6. Real-Time Synchronisation Implementation 24

6. Testing & Results 31

6.1. Authentication Testing 31

6.2. Firestore CRUD Testing 32
6.3. Security Rules Testing 32
7. Limitations 33
8. Conclusion & Future Scope 36
8.1. Conclusion 36
8.2. Future Scope 37
9. Bibliography 39
9.1. A.1 Firebase Security Rules (Complete) 40
9.2. A.2 Admin SDK — Role Assignment Script (set-role.js) 44
9.3. A.3 Admin SDK — Firestore Seed Script (seed.js) 49
9.4. A.4 Firebase Hosting Configuration (firebase.json) 51
9.5. A.5 Kotlin — Firebase Auth with Role Redirect 51
9.6. A.6 JavaScript — Real-Time Assessments Listener 54

LIST OF TABLES

Table No.	Name of Table	Page No.
Table 1	Team Member Roles and Responsibilities	9
Table 2	Firebase Services Free Tier Limits and Project Usage	13
Table 3	Backend Architecture Components and Technologies	16
Table 4	Users Collection Schema Fields	21
Table 5	Patients Collection Schema Fields	22
Table 6	Quiz Results Subcollection Schema Fields	22
Table 7	Assessments Subcollection Schema Fields	22
Table 8	Stages Collection Schema Fields	23
Table 9	Videos Subcollection Schema Fields	23
Table 10	Firestore Security Rules Access Matrix by Role	31
Table 11	Authentication Test Cases and Results	39
Table 12	Firestore Operations Test Cases and Results	40
Table 13	Security and Validation Test Cases and Results	40
Table 14	Abbreviations and Acronyms	67

LIST OF IMAGES

Figure No.	Name of Figure	Page No.
Figure 1	ER Diagram	18
Figure 2	DFD Level 0	19
Figure 3	DFD Level 1	20
Figure 4	Firestore Collection Hierarchy	23
Figure 5	Firebase Auth Flow Diagram	24

1. Introduction

1.1. Abstract of Project

Sound for Silence is a dual-interface digital healthcare platform designed for children undergoing cochlear implant rehabilitation. The system comprises an Android mobile application for patients and parents, and a web-based dashboard for therapists and clinicians, unified by a Firebase cloud backend.

This individual report documents the work carried out in the role of **Backend & Firebase Engineer**. The responsibilities of this role encompassed the complete design and implementation of the Firebase backend infrastructure — including Firebase Authentication with role-based access control, Cloud Firestore data modelling and schema design, Firebase Security Rules, and Firebase Hosting for the web dashboard. This backend layer serves as the single source of truth and the data synchronisation bridge between the Android application and the therapist web portal.

The project was developed as a group of four members, each with a distinct technical responsibility. The group roles are as follows:

1	Android Developer	Kotlin + Jetpack Compose UI, ExoPlayer video playback, quiz logic, Firebase Android SDK integration
2	Web Dashboard Developer	HTML/CSS/JS therapist portal, Chart.js progress charts, Firebase JS SDK integration
3	Backend & Firebase Engineer **(This Report)**	Firebase Auth, Firestore schema, Security Rules, Firebase Hosting, backend architecture
4	UI/UX Design & Testing	Wireframes, screen layouts, test case design, user acceptance testing, documentation

Table 1: Team Member Roles and Responsibilities

1.1.1. Objective

The objectives of Sound for Silence as a system are:

- To develop an Android application delivering sequential, gamified therapy content to children with cochlear implants, tracking learning progress automatically.

- To build a web-based therapist dashboard allowing clinicians to manage patient records, record clinical scores (CAP and SIR), and visualize long-term progress using interactive charts.
- To implement a unified Firebase backend providing secure, role-based authentication and real-time data synchronisation between the Android app and the web dashboard without requiring a dedicated server.
- To design a parent-facing portal providing clear visibility into their child's daily activity, quiz performance, and overall rehabilitation progress.

The specific objectives of the Backend & Firebase Engineer role are:

- To design a normalised, scalable Firestore data model covering all entities: users, patients, therapy stages, videos, quiz results, and clinical assessments.
- To implement Firebase Authentication with email/password login and custom-claim-based role enforcement for three user types: parent, therapist, and admin.
- To write comprehensive Firestore Security Rules that enforce role-based data access at the database level, preventing cross-role data exposure.
- To configure Firebase Hosting for the web dashboard with a clean deployment pipeline.
- To ensure real-time synchronisation of data between the Android app and the web portal via Firestore listeners.

1.2. Problem Specification / Need of Project

Children who receive cochlear implants require months to years of structured auditory and speech rehabilitation. The therapy protocol involves progressive stages of auditory training, regular quizzes to assess comprehension, and periodic clinical assessments using standardised scoring tools such as the CAP (Categories of Auditory Performance) and SIR (Speech Intelligibility Rating) scales.

In current practice, this rehabilitation process suffers from several critical gaps. Therapy progress is commonly tracked using paper records maintained by therapists, making longitudinal monitoring difficult, error-prone, and inaccessible to parents between clinic visits. There is no structured mechanism for parents to monitor their child's daily engagement with therapy exercises. Therapists lack a digital tool to view historical CAP and SIR score trends across months, making evidence-based clinical decisions harder.

From a backend perspective, the core problem is the absence of a centralised, real-time data store that securely connects the child's in-app activity with the therapist's clinical dashboard. Any platform attempting to solve this problem requires a backend capable of: authenticating three distinct user roles without cross-contamination of data access, storing hierarchical data (user → patient → stage → video → quiz → assessment), synchronising updates across Android and web in real time, and enforcing fine-grained access control without a dedicated application server.

Sound for Silence addresses all these requirements using Firebase as a serverless backend, making it operationally practical, cost-effective, and scalable for clinical environments.

2. Feasibility Study

2.1. Technical Feasibility

The backend infrastructure of Sound for Silence is built entirely on Firebase, a managed Backend-as-a-Service (BaaS) platform provided by Google. Firebase eliminates the requirement for a dedicated application server, reducing operational complexity to zero while providing production-grade cloud infrastructure.

Firebase Authentication supports email/password authentication natively, with built-in token management, session persistence, and secure password hashing. Custom claims can be set via the Firebase Admin SDK to encode user roles (parent, therapist) directly into the JWT token, enabling role enforcement both on the client and in Security Rules without an additional identity service.

Cloud Firestore is a NoSQL document database with native real-time synchronisation over WebSocket connections. It supports offline persistence on Android via the Firebase Android SDK, meaning the app can continue to function and sync when connectivity is restored. Firestore's hierarchical document-collection model maps naturally to the data structure required by this project.

Firebase Security Rules is a declarative rules language that enforces access control at the database layer, independent of application code. Rules can inspect the authenticated user's UID, custom claims, and document fields to allow or deny reads and writes. This provides server-side security enforcement without a custom backend.

Firebase Hosting provides CDN-backed static hosting for the web dashboard, with a one-command deployment via the Firebase CLI. HTTPS is enforced by default.

All of these services are well-documented, actively maintained by Google, and have been used in production healthcare and education applications. The technical complexity of the backend implementation is well within the scope of a final-year B.Tech project.

2.2. Operational Feasibility

From an operational standpoint, the Firebase backend is transparent to end users. Parents using the Android app and therapists using the web dashboard interact with their respective interfaces without any awareness of the underlying backend. Data entered by

the therapist appears in the parent's view within seconds via Firestore real-time listeners, without any manual refresh or polling.

The serverless nature of Firebase means there are no servers to manage, update, or monitor. Firebase Console provides a visual interface for monitoring authentication events, database reads and writes, hosting deployments, and security rule evaluations. This makes the backend operationally manageable by non-DevOps personnel such as a college faculty member overseeing the project.

2.3. Economic Feasibility

Firebase's Spark (free) plan covers all backend requirements for this project:

Service	Free Tier Limit	Project Usage
Firebase Authentication	Unlimited (email/pass-word)	< 100 users for develop-ment
Cloud Firestore	50,000 reads/day, 20,000 writes/day, 1 GB storage	Sufficient for develop-ment and demo
Firebase Hosting	10 GB storage, 360 MB/ day transfer	Well within limit for dash-board
Firebase Admin SDK	Free (runs client-side or via Cloud Functions)	Used for custom claims setup

Table 2: Firebase Services Free Tier Limits and Project Usage

No dedicated server, VPS, or paid database subscription is required. The total monetary cost of the backend is zero, making it economically ideal for a student project.

3. Software Requirement Specifications

3.1. Introduction

Backend Software Requirements Overview: The backend is the authoritative integration layer connecting the Android app and web dashboard, responsible for data management, identity, access control, integrity, and coordinated state across clients.

Scope of Backend Responsibilities: Manages secure authentication and fine-grained authorization for Parents, Therapists, and Admins. Persists patients, assessments, sessions, media, quizzes, and telemetry in a scalable NoSQL schema with subcollections; supports real-time sync and declarative, role-aware security rules. Implements serverless business logic (Cloud Functions) for event-driven automation and Hosting for the therapist portal.

Data Integrity, Observability, and Compliance Considerations: Maintains audit trails and append-only logs for sensitive events and supports indexed, analytics-friendly document shapes for reporting. Provides export options (CSV/NDJSON, FHIR endpoints) and enforces encryption in transit/at rest; recommends client-side encryption for highly sensitive media and BAA/dedicated FHIR stores where required.

Operational Resilience and Cost Efficiency: Uses serverless Firebase primitives (Firestore, Cloud Functions, Hosting) for horizontal scalability and low ops overhead. Includes rate-limiting, per-user quotas, monitoring, CI/CD-driven deployments, staging/production segregation, and documented migration paths to control costs.

User Roles and Interaction Patterns: Parents use the Android app to view/submit progress and consume content; Therapists use the web dashboard to manage patients, configure plans, and run reports; Admins perform provisioning and emergency remediation via Admin SDK/Console. Role-specific APIs and rules ensure isolation and least-privilege access.

Developer Experience, Testing, and Lifecycle Management: Automated test suites validate Firestore security rules and CRUD scenarios; CI/CD pipelines test Cloud Functions and schema migrations in staging before production. Includes documentation, automated backups, migration procedures, and observability tooling to support audits and safe evolution.

3.2. Selection of Technology / Specific Requirements

3.2.1. Functional Requirements

Authentication:

- The system shall authenticate users via email and password using Firebase Authentication.
- The system shall assign a role (parent or therapist) to each user account using Firebase Custom Claims.
- The system shall maintain session persistence across app restarts for authenticated users.
- The system shall reject unauthenticated requests to all Firestore data.

Data Storage and Retrieval:

- The system shall store all user profile data, patient records, therapy stage metadata, video metadata, quiz results, and clinical assessments in Cloud Firestore.
- The system shall support real-time listeners so that data changes in Firestore are pushed to connected clients within 2 seconds under normal network conditions.
- The Android app shall support offline data access via Firestore's local persistence cache.

Access Control:

- A parent shall only be able to read data for their own child's patient record, stage progress, and quiz results. They shall not be able to read or write therapist data.
- A therapist shall be able to read and write clinical assessment records (CAP/SIR scores) for their assigned patients only.
- No user shall be able to modify another user's authentication profile or custom claims from the client.
- Security Rules shall enforce all of the above at the Firestore layer, independent of application code.

Hosting:

- The web dashboard shall be deployed to Firebase Hosting and accessible via HTTPS.
- The hosted dashboard shall load within 3 seconds on a standard broadband connection.

3.2.2. Non-Functional Requirements

- **Security:** All data in transit is encrypted via HTTPS/TLS. Firestore data at rest is encrypted by Google. Passwords are never stored in plaintext — Firebase Authentication handles hashing internally.
- **Scalability:** Firestore scales horizontally without configuration. The backend design shall not include any hard-coded limits that would prevent scaling to hundreds of patients.
- **Reliability:** Firebase provides 99.95% uptime SLA. No custom reliability engineering is required.
- **Maintainability:** The Firestore schema and Security Rules shall be documented clearly. All collection and field names shall follow a consistent camelCase naming convention.

3.2.3. Technology Stack

Component	Technology	Purpose
Authentication	Firebase Authentication	Email/password login, JWT tokens, custom role claims
Database	Cloud Firestore (NoSQL)	All persistent application data storage and real-time sync
Security	Firestore Security Rules	Role-based access control at database layer
Hosting	Firebase Hosting	Static deployment of web dashboard with CDN and HTTPS
Admin Operations	Firebase Admin SDK (Node.js)	Setting custom claims, seeding initial data
Client SDK — Android	Firebase Android SDK (BOM 32.x)	Auth and Firestore access from Kotlin app
Client SDK — Web	Firebase JS SDK v9 (Modular)	Auth and Firestore access from JS dashboard
Project Config	google-services.json	Android Firebase project configuration

Table 3: Backend Architecture Components and Technologies

4. Design

4.1. ER Diagram

The Entity-Relationship diagram models the logical data entities in Sound for Silence and their relationships, prior to mapping them to Firestore's document-collection structure.

Entities and Attributes:

- **User** — uid (PK), email, role (parent | therapist), displayName, createdAt
- **Parent** — parentId (FK → User.uid), childPatientId (FK → Patient)
- **Therapist** — therapistId (FK → User.uid), assignedPatientIds[]
- **Patient** — patientId (PK), name, dateOfBirth, implantDate, cochlearDevice, assignedTherapistId (FK → Therapist)
- **Stage** — stageId (PK), title, order, isLocked (computed)
- **Video** — videoId (PK), stageId (FK → Stage), title, videoUrl, order
- **QuizResult** — resultId (PK), patientId (FK → Patient), videoId (FK → Video), score, totalQuestions, completedAt
- **ClinicalAssessment** — assessmentId (PK), patientId (FK → Patient), therapistId (FK → Therapist), capScore (0–7), sirScore (0–5), assessmentDate, notes

Key Relationships:

- One Parent → One Patient (one-to-one)
- One Therapist → Many Patients (one-to-many)
- One Patient → Many QuizResults (one-to-many)
- One Patient → Many ClinicalAssessments (one-to-many)
- One Stage → Many Videos (one-to-many)
- One Video → Many QuizResults (one-to-many)

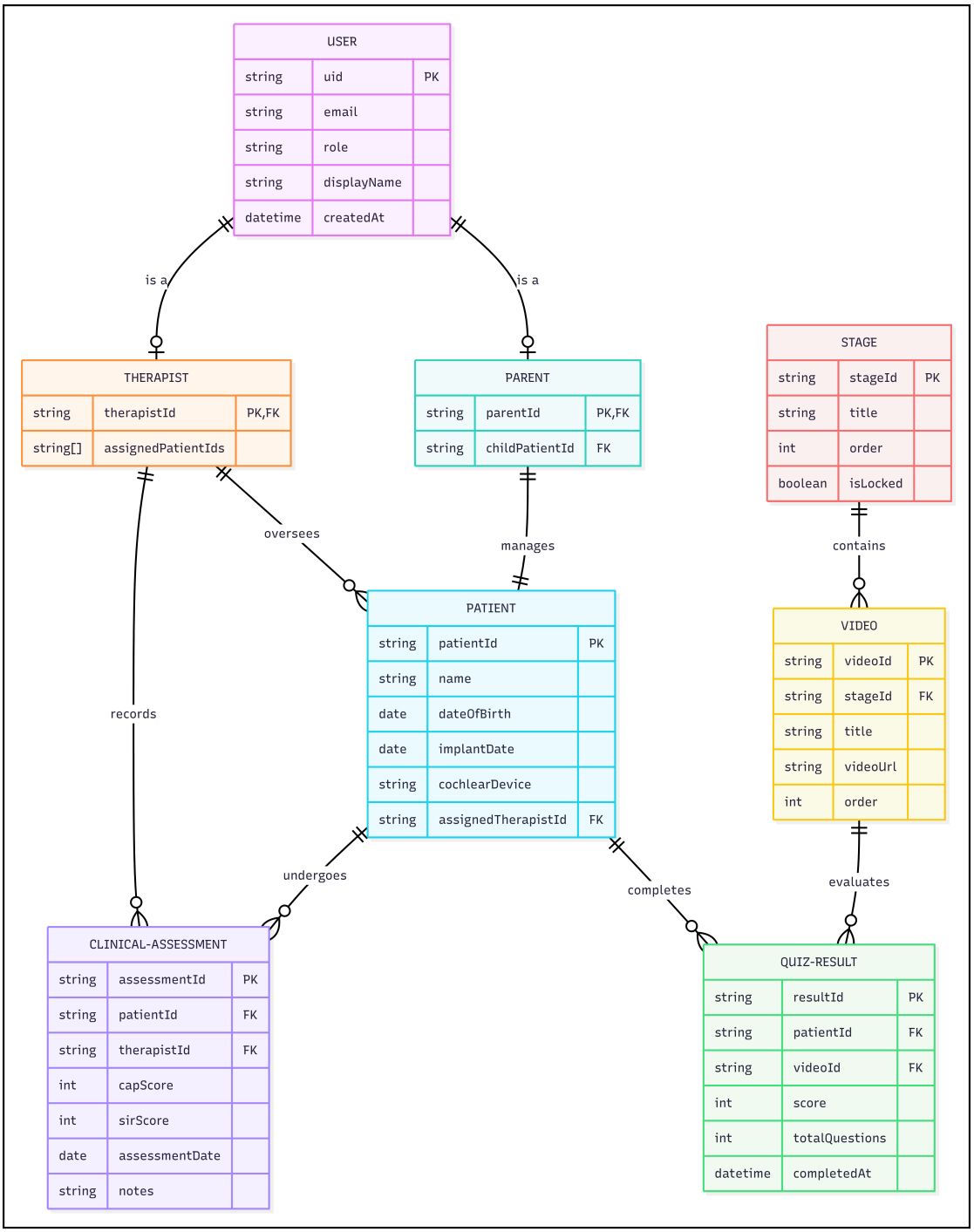

Figure 1: ER Diagram

4.2. Data Flow Diagram (Level 0 & Level 1)

4.2.1. Level 0 — Context Diagram

The Level 0 DFD shows the system as a single process receiving inputs from three external entities and producing outputs back to them.

External Entities: Parent (via Android app), Therapist (via web dashboard), Firebase Cloud (external BaaS)

Data Flows:

- Parent → System: Login credentials, quiz answers, video completion events
- System → Parent: Auth token, stage content, progress data, quiz results
- Therapist → System: Login credentials, CAP/SIR scores, patient notes
- System → Therapist: Auth token, patient list, assessment history, progress charts
- System ↔ Firebase Cloud: All reads/writes pass through Firestore and Auth APIs

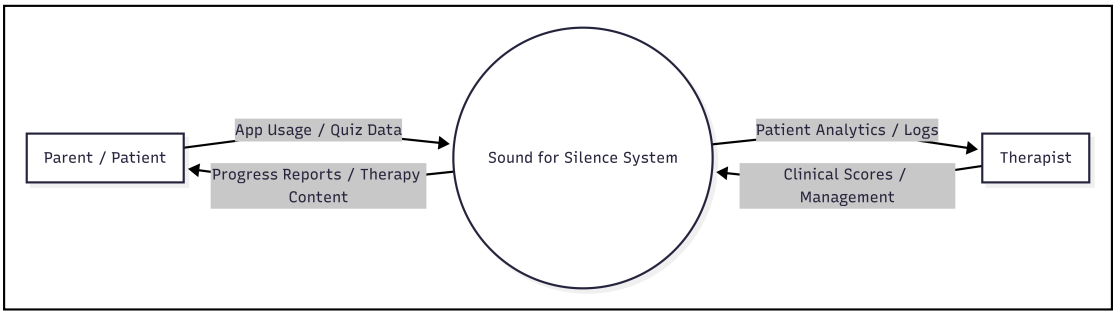

Figure 2: DFD Level 0

4.2.2. Level 1 — Expanded DFD

At Level 1, the system is decomposed into five major processes:

1. **Authentication Process** — Accepts credentials from parent or therapist, sends them to Firebase Auth, receives JWT with role claims, returns session to client.
2. **Stage & Content Management** — Reads stage and video metadata from Firestore `stages` and `videos` collections; applies unlock logic based on `quizResults` for the patient.
3. **Quiz Processing** — Accepts quiz answers from the Android app, computes score, writes `QuizResult` document to Firestore under the patient's subcollection.
4. **Clinical Score Recording** — Accepts CAP/SIR input from therapist dashboard, validates score range (CAP: 0–7, SIR: 0–5), writes `assessments` document to Firestore.
5. **Progress Data Aggregation** — Reads `quizResults` and `assessments` for a patient, aggregates data for Chart.js rendering in the therapist dashboard and summary stats in the parent portal.

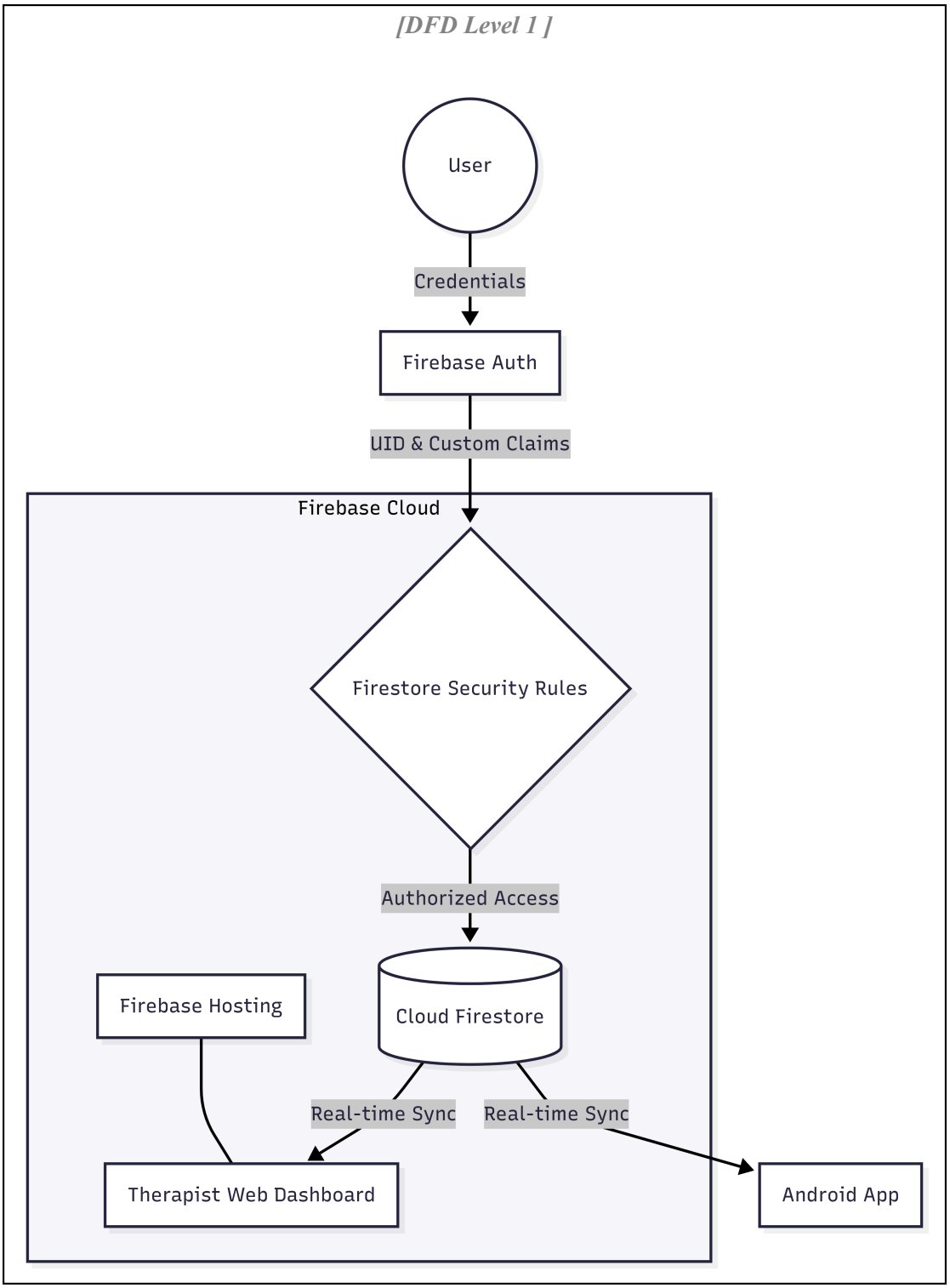

Figure 3: DFD Level 1

4.3. Modules

From the backend perspective, the system is organised into four modules:

4.3.1. Module 1: Authentication & Role Management

This module handles all user identity operations. Firebase Authentication manages account creation, login, session tokens, and password management. A one-time Node.js

Admin SDK script sets the `role` custom claim on each user account after creation. The claim is read by Firestore Security Rules on every request to determine the permitted operations.

4.3.2. Module 2: Firestore Data Layer

This module is the core persistence layer. It defines the collection hierarchy and manages all reads and writes from both the Android SDK and the Firebase JS SDK. The data layer is stateless — it has no server-side business logic. All data validation is handled either by client-side code (Android/JS) or by Security Rules field constraints.

4.3.3. Module 3: Security Rules Engine

This module is the backend's access control layer. Written in the Firestore Security Rules language, it evaluates every read and write request against the authenticated user's UID and role claims before allowing or denying the operation. This runs entirely within Firebase infrastructure and requires no server.

4.3.4. Module 4: Firebase Hosting & Deployment

This module manages the deployment of the web dashboard to Firebase Hosting. The Firebase CLI (`firebase deploy`) uploads the static files (HTML, CSS, JS) to Firebase's CDN. A `firebase.json` configuration file defines the hosting root directory and URL rewrite rules.

4.4. Database

The Firestore database is structured as a hierarchy of collections and documents. Below is the complete schema:

4.4.1. Collection: `users`

Document ID: `{uid}` (Firebase Auth UID)

Field	Type	Description
uid	string	Firebase Auth UID (same as document ID)
email	string	User's email address
role	string	"parent" or "therapist"
displayName	string	Full name
createdAt	timestamp	Account creation timestamp

Table 4: Users Collection Schema Fields

4.4.2. Collection: `patients`

Document ID: `{patientId}` (auto-generated)

Field	Type	Description
name	string	Child's full name
dateOfBirth	timestamp	Date of birth
implantDate	timestamp	Date of cochlear implant surgery
cochlearDevice	string	Device model/manufacturer
parentUid	string	UID of the parent user
therapistUid	string	UID of the assigned therapist
currentStage	number	Index of the current therapy stage (0-based)
createdAt	timestamp	Record creation timestamp

Table 5: Patients Collection Schema Fields

4.4.3. Subcollection: `patients/{patientId}/quizResults`

Field	Type	Description
videoId	string	ID of the video this quiz belongs to
stageId	string	ID of the stage
score	number	Number of correct answers
totalQuestions	number	Total questions in the quiz
completedAt	timestamp	Quiz completion timestamp
passed	boolean	true if score ≥ passing threshold

Table 6: Quiz Results Subcollection Schema Fields

4.4.4. Subcollection: `patients/{patientId}/assessments`

Field	Type	Description
therapistUid	string	UID of the therapist who recorded the score
capScore	number	CAP score, integer in range 0–7
sirScore	number	SIR score, integer in range 0–5
assessmentDate	timestamp	Date of clinical session
notes	string	Optional clinical notes (max 500 chars)

Table 7: Assessments Subcollection Schema Fields

4.4.5. Collection: `stages`

Document ID: `{stageId}`

Field	Type	Description
title	string	Stage title (e.g., "Stage 1: Detection")
order	number	Sequential order index (1, 2, 3 …)
description	string	Brief description of the stage's therapy goals

Table 8: Stages Collection Schema Fields

4.4.6. Subcollection: `stages/{stageId}/videos`

Field	Type	Description
title	string	Video title
videoUrl	string	URL of the hosted video (Firebase Storage or CDN)
order	number	Order within the stage
quizQuestions	array	Array of quiz question objects (question, options, correctIndex)

Table 9: Videos Subcollection Schema Fields

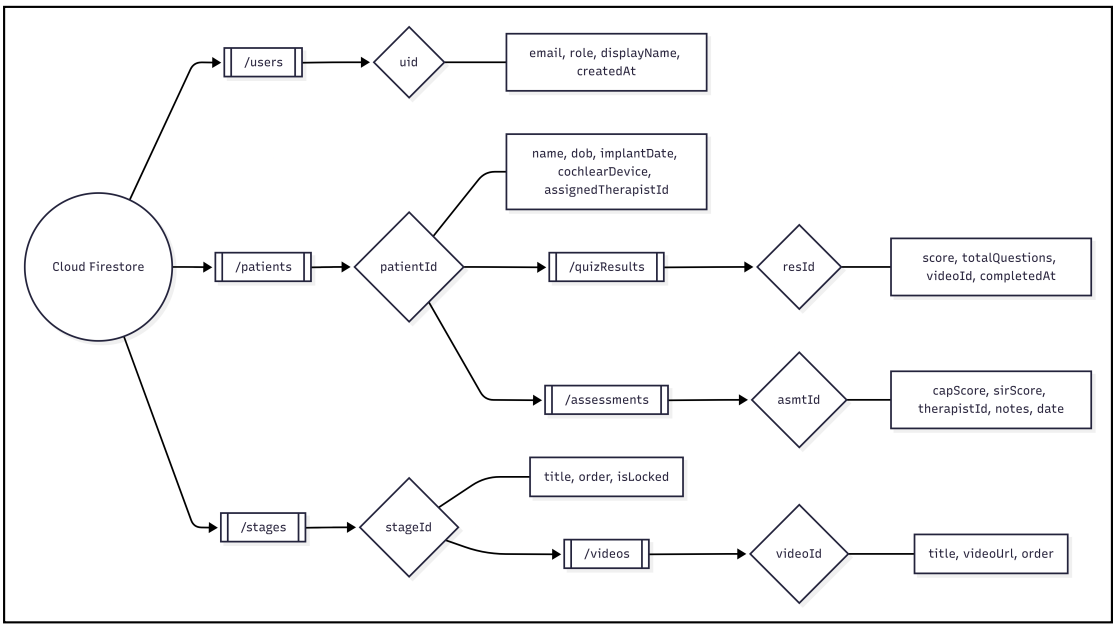

Figure 4: Firestore Collection Hierarchy

4.5. Input–Output Form / Screen Layout

From the backend perspective, inputs and outputs are API calls (Firestore reads/writes and Auth requests) rather than visual screens. The relevant I/O interactions are:

Input: Login Request

- Input fields: email (string), password (string), role selection (parent | therapist)
- Output: Firebase Auth JWT token with custom claim `role`, user UID, displayName

Input: Quiz Result Submission (Android)

- Input: patientId, videoId, stageId, score, totalQuestions, completedAt
- Backend action: Write document to `patients/{patientId}/quizResults`
- Output: Document reference ID, Firestore write acknowledgement

Input: Clinical Assessment Entry (Web Dashboard)

- Input: patientId, capScore (0–7), sirScore (0–5), assessmentDate, notes
- Backend action: Validate field ranges via Security Rules, write document to `patients/{patientId}/assessments`
- Output: Document reference ID or rejection reason

Output: Patient Progress Query (Web Dashboard)

- Input: therapistUid (from JWT), patientId
- Firestore query: `patients/{patientId}/assessments` ordered by `assessmentDate` ascending
- Output: Array of assessment documents for Chart.js rendering

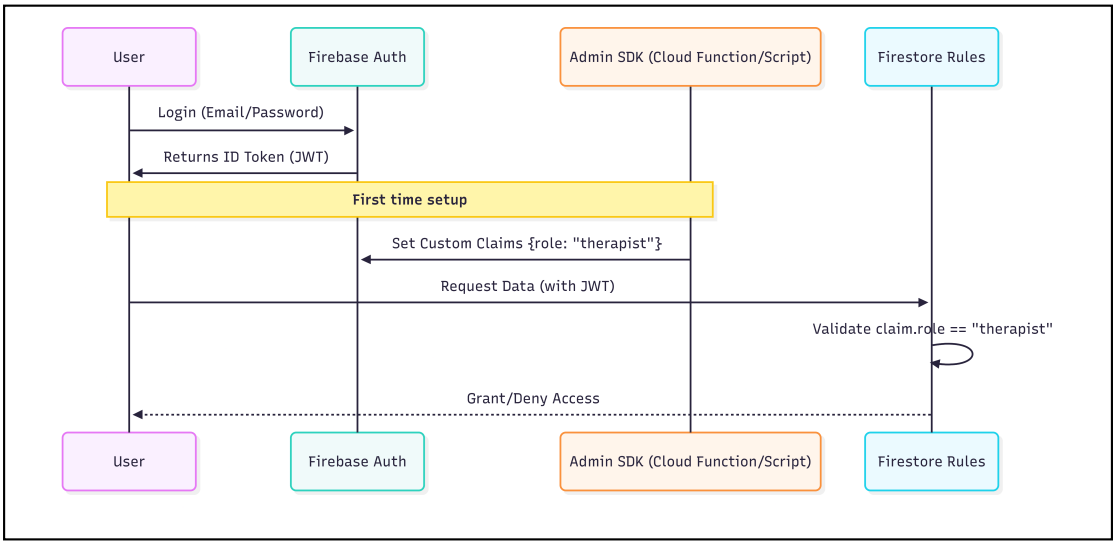

Figure 5: Firebase Auth Flow Diagram

5. Implementation / Technological Environment

This chapter documents the complete backend implementation carried out as part of the Backend & Firebase Engineer role.

5.1. Firebase Project Setup

Firebase Project & Services: A single Firebase project at console.firebase.google.com was used as the unified backend for both Android and web, with Authentication (email/ password), Cloud Firestore (production mode), and Firebase Hosting enabled.

Android App Setup: Android app registered in the project; downloaded google-services.json placed in app/ to provide credentials and project identifiers enabling auth, Firestore access, and Firebase features.

Web Dashboard Setup: Web app registered in the same project; firebaseConfig object (API keys and project IDs) embedded in the dashboard JS to enable auth and Firestore access.

Benefits of the Unified Architecture: Centralized data management (single Firestore) for consistency; cross-platform consistency and shared authentication; simplified deployment via Firebase Hosting.

5.2. Firebase Authentication Implementation

Firebase Authentication was configured with the **Email/Password** sign-in provider. Two user types — parents and therapists — are distinguished via **Custom Claims**, which are metadata fields attached to the Firebase Auth JWT token.

Custom claims are set using the Firebase Admin SDK (Node.js), executed once during account provisioning:

```
// set-roles.js — minimal, safe Firebase Auth custom-claims updater
const admin = require("firebase-admin");
const fs = require("fs");
const readline = require("readline");

admin.initializeApp();

const MAX_RETRIES = 3;
```

```
const RETRY_BASE_MS = 500;

const log = (...a) => console.log(new Date().toISOString(), ...a);
const wait = (ms) => new Promise((r) => setTimeout(r, ms));

async function retry(fn) {
  for (let i = 0; ; i++) {
    try { return await fn(); }
    catch (e) {
      if (i >= MAX_RETRIES) throw e;
      await wait(RETRY_BASE_MS * 2 ** i);
    }
  }
}

async function readCsv(file) {
  const rl = readline.createInterface({ input:
fs.createReadStream(file), crlfDelay: Infinity });
  const rows = [];
  let n = 0;
  for await (const line of rl) {
    n++;
    if (n === 1 && line.toLowerCase().startsWith("uid")) continue;
    if (!line.trim()) continue;
    const [uid, role] = line.split(",").map(s => s && s.trim());
    if (uid && role) rows.push({ uid, role });
  }
  return rows;
}

async function ensureRole(uid, role, dry) {
  return retry(async () => {
    const user = await admin.auth().getUser(uid);
    const cur = (user.customClaims && user.customClaims.role) || null;
    if (cur === role) { log(`noop ${uid} already=${role}`); return; }
    if (dry) { log(`[dry] would set ${uid} -> ${role} (was ${cur})`);
```

```
return; }
    await admin.auth().setCustomUserClaims(uid, { ...
(user.customClaims||{}), role });
    log(`updated ${uid} -> ${role}`);
  });
}

async function run() {
  const args = process.argv.slice(2);
  const dry = args.includes("--dry-run") || args.includes("-n");
  const file = args.find(a => a.endsWith(".csv"));
  const assignments = file ? await readCsv(file) : [
    { uid: "therapist_uid_here", role: "therapist" },
    { uid: "parent_uid_here", role: "parent" },
  ];
  log(`starting ${assignments.length} items dry=${dry}`);
  for (const { uid, role } of assignments) {
    try { await ensureRole(uid, role, dry); }
    catch (e) { log(`error ${uid}:`, e.message || e); }
  }
  log("done");
  process.exit(0);
}

if (require.main === module) run().catch(e => { console.error(e);
process.exit(1); });
```

On the Android client, the custom claim is read from the token after login:

```
// Kotlin — read role from Firebase ID token
FirebaseAuth.getInstance().currentUser?.getIdToken(false)
  ?.addOnSuccessListener { result ->
    val role = result.claims["role"] as? String
    // Navigate to appropriate screen based on role
    if (role == "parent") navigateToParentDashboard()
  }
```

On the web client, the same token is read via the Firebase JS SDK:

```
// JavaScript — read role claim after login
import { getAuth } from "firebase/auth";

const auth = getAuth();
auth.currentUser.getIdTokenResult().then((idTokenResult) => {
  const role = idTokenResult.claims.role;
  if (role === "therapist") renderTherapistDashboard();
});
```

5.3. Cloud Firestore — Data Seeding

Initial stage and video data was seeded into Firestore using the Firebase Admin SDK. A `seed.js` script was written to create the `stages` collection and their `videos` subcollections with the correct structure:

```
const admin = require("firebase-admin");
const db = admin.firestore();

const stages = [
  {
    id: "stage_01",
    title: "Stage 1: Sound Detection",
    order: 1,
    description: "Learning to detect the presence and absence of
sound.",
  },
  {
    id: "stage_02",
    title: "Stage 2: Sound Discrimination",
    order: 2,
    description: "Distinguishing between different environmental
sounds.",
  },
  // ... additional stages
];
```

```
async function seedStages() {
  for (const stage of stages) {
    await db.collection("stages").doc(stage.id).set({
      title: stage.title,
      order: stage.order,
      description: stage.description,
    });
    console.log(`Seeded stage: ${stage.id}`);
  }
}

seedStages();
```

5.4. Firestore Security Rules Implementation

The Security Rules are the core of the backend access control system. They are written in the Firestore Rules language and deployed via the Firebase CLI. The complete implemented rules are as follows:

```
// Helpers
function isAuthenticated() { return request.auth != null; }
function getRole() { return request.auth.token.role; }
function isParent() { return isAuthenticated() && getRole() ==
"parent"; }
function isTherapist() { return isAuthenticated() && getRole() ==
"therapist"; }
function isOwnerOfPatient(patientId) {
  return isAuthenticated() &&
    get(/databases/\$(database)/documents/patients/\
$(patientId)).data.parentUid == request.auth.uid;
}
function isAssignedTherapist(patientId) {
  return isAuthenticated() &&
    get(/databases/\$(database)/documents/patients/\
$(patientId)).data.therapistUid == request.auth.uid;
}

// users — only owners
```

```
match /users/{uid} {
  allow read, write: if isAuthenticated() && request.auth.uid == uid;
}

// patients
match /patients/{patientId} {
  allow read: if isParent() && isOwnerOfPatient(patientId);
  allow read, write: if isTherapist() &&
isAssignedTherapist(patientId);

  // quizResults
  match /quizResults/{resultId} {
    allow read: if isParent() && isOwnerOfPatient(patientId);
    allow write: if isParent() && isOwnerOfPatient(patientId);
    allow read: if isTherapist() && isAssignedTherapist(patientId);
  }

  // assessments
  match /assessments/{assessmentId} {
    allow read: if isTherapist() && isAssignedTherapist(patientId);
    allow read: if isParent() && isOwnerOfPatient(patientId);
    allow write: if isTherapist() && isAssignedTherapist(patientId)
                 && request.resource.data.capScore is int
                 && request.resource.data.capScore >= 0
                 && request.resource.data.capScore <= 7
                 && request.resource.data.sirScore is int
                 && request.resource.data.sirScore >= 0
                 && request.resource.data.sirScore <= 5;
  }
}

// stages and videos — read-only for authenticated users
match /stages/{stageId} {
  allow read: if isAuthenticated();
  allow write: if false;
  match /videos/{videoId} {
```

```
    allow read: if isAuthenticated();
    allow write: if false;
  }
}
```

The rules enforce the following access matrix:

Collection	Parent	Therapist	Admin SDK
users/{own}	R/W	R/W	R/W
patients (own child)	R	R/W (assigned)	R/W
patients/quizResults	R/W	R	R/W
patients/assessments	R	R/W (validated)	R/W
stages	R	R	R/W
stages/videos	R	R	R/W

Table 10: Firestore Security Rules Access Matrix by Role

5.5. Firebase Hosting — Deployment

The web dashboard was deployed to Firebase Hosting using the Firebase CLI. The `firebase.json` configuration file defines the hosting settings:

```
{
  "hosting": {
    "public": "public",
    "ignore": [
      "firebase.json",
      "**/.*",
      "**/node_modules/**"
    ],
    "rewrites": [
      {
        "source": "**",
        "destination": "/index.html"
      }
    ]
  }
}
```

Deployment was performed using:

```
firebase login
firebase init hosting
firebase deploy --only hosting
```

After deployment, the dashboard was accessible at the Firebase-assigned domain (`sound-for-silence.web.app`) over HTTPS, with automatic SSL certificate provisioning by Firebase.

5.6. Real-Time Synchronisation Implementation

Firestore real-time listeners were implemented on both clients to push data updates without polling. On the web dashboard, the therapist's patient list updates in real time:

```
// JavaScript — real-time listener for assigned patients with enhanced
functionality
import {
  collection,
  query,
  where,
  onSnapshot,
  orderBy,
  limit,
} from "firebase/firestore";

class TherapistPatientManager {
  constructor(db, auth) {
    this.db = db;
    this.auth = auth;
    this.unsubscribe = null;
    this.patients = new Map();
    this.isListening = false;
  }

  /**
   * Starts real-time listener for assigned patients
   * @param {Function} onPatientsUpdate - Callback when patients data
changes
```

```
   * @param {Function} onError - Callback for error handling
   * @returns {Function} - Unsubscribe function
   */
  startListening(onPatientsUpdate, onError) {
    try {
      const therapistUid = this.auth.currentUser?.uid;

      if (!therapistUid) {
        throw new Error("No authenticated user found");
      }

      const q = query(
        collection(this.db, "patients"),
        where("therapistUid", "==", therapistUid),
        orderBy("createdAt", "desc"),
        limit(100)
      );

      this.unsubscribe = onSnapshot(
        q,
        (snapshot) => {
          this.handleSnapshot(snapshot, onPatientsUpdate);
        },
        (error) => {
          this.handleError(error, onError);
        }
      );

      this.isListening = true;
      console.log("Patient listener started successfully");

      return this.unsubscribe;
    } catch (error) {
      console.error("Failed to start patient listener:",
error.message);
      if (onError) onError(error);
```

```
    return null;
  }
}

/**
 * Handles snapshot updates from Firestore
 * @param {QuerySnapshot} snapshot - Firestore query snapshot
 * @param {Function} onPatientsUpdate - Callback to update UI
 */
handleSnapshot(snapshot, onPatientsUpdate) {
  const changes = {
    added: [],
    modified: [],
    removed: [],
  };

  snapshot.docChanges().forEach((change) => {
    const patientId = change.doc.id;
    const patientData = change.doc.data();

    switch (change.type) {
      case "added":
        this.patients.set(patientId, patientData);
        changes.added.push({ id: patientId, data: patientData });
        console.log(`Patient added: ${patientId}`);
        break;

      case "modified":
        this.patients.set(patientId, patientData);
        changes.modified.push({ id: patientId, data: patientData });
        console.log(`Patient modified: ${patientId}`);
        break;

      case "removed":
        this.patients.delete(patientId);
        changes.removed.push(patientId);
```

```
          console.log(`Patient removed: ${patientId}`);
          break;
        }
      });

      // Only call update if there are actual changes
      if (
        changes.added.length > 0 ||
        changes.modified.length > 0 ||
        changes.removed.length > 0
      ) {
        onPatientsUpdate({
          patients: Array.from(this.patients.entries()).map(([id, data])
=> ({
            id,
            ...data,
          })),
          changes,
        });
      }
    }

    /**
     * Handles errors from the listener
     * @param {Error} error - The error object
     * @param {Function} onError - Error callback
     */
    handleError(error, onError) {
      console.error("Patient listener error:", error.message);

      if (error.code === "permission-denied") {
        console.error("Permission denied: Check Firestore rules");
      } else if (error.code === "unavailable") {
        console.warn("Firestore temporarily unavailable, retrying...");
      }
```

```
  if (onError) onError(error);
}

/**
 * Stops the real-time listener
 */
stopListening() {
  if (this.unsubscribe) {
    this.unsubscribe();
    this.isListening = false;
    this.patients.clear();
    console.log("Patient listener stopped");
  }
}

/**
 * Gets all currently tracked patients
 * @returns {Array} - Array of patient objects
 */
getPatients() {
  return Array.from(this.patients.entries()).map(([id, data]) => ({
    id,
    ...data,
  }));
}

/**
 * Gets a specific patient by ID
 * @param {string} patientId - The patient's document ID
 * @returns {Object|null} - Patient data or null
 */
getPatient(patientId) {
  return this.patients.get(patientId) || null;
}

/**
```

```
   * Gets patient count
   * @returns {number} - Number of assigned patients
   */
  getPatientCount() {
    return this.patients.size;
  }
}

// Usage example
const patientManager = new TherapistPatientManager(db, auth);

patientManager.startListening(
  (update) => {
    const { patients, changes } = update;

    // Handle added patients
    changes.added.forEach((patient) => {
      renderPatientCard(patient.id, patient.data);
    });

    // Handle modified patients
    changes.modified.forEach((patient) => {
      updatePatientCard(patient.id, patient.data);
    });

    // Handle removed patients
    changes.removed.forEach((patientId) => {
      removePatientCard(patientId);
    });

    // Update patient count
    updatePatientCount(patients.length);
  },
  (error) => {
    console.error("Failed to load patients:", error);
    showErrorNotification("Could not load patient list");
```

```
  }
);

// Clean up when component unmounts
// patientManager.stopListening();
```

On Android (Kotlin), a similar listener updates the parent's progress view:

```
// Kotlin — real-time listener for patient progress
val patientRef = db.collection("patients").document(patientId)

patientRef.addSnapshotListener { snapshot, error ->
  if (error != null || snapshot == null) return@addSnapshotListener
  val currentStage = snapshot.getLong("currentStage") ?: 0L
  updateParentDashboard(currentStage.toInt())
}
```

6. Testing & Results

Backend testing was conducted across three areas: Authentication, Firestore CRUD operations, and Security Rules validation.

6.1. Authentication Testing

TC	Test Case	Input	Expected Output	Result
TC01	Valid parent login	Correct email + password (role: parent)	JWT with role="parent", redirect to parent dashboard	Pass
TC02	Valid therapist login	Correct email + password (role: therapist)	JWT with role="therapist", redirect to therapist dashboard	Pass
TC03	Invalid password	Correct email + wrong password	Auth error	Pass
TC04	Unregistered email	Random email + password	Auth error	Pass
TC05	Session persistence	Close and reopen app after login	User remains logged in, no re-login required	Pass
TC06	Custom claim present	Login as therapist, fetch ID token	Token contains `role: "therapist"` in claims	Pass

Table 11: Authentication Test Cases and Results

6.2. Firestore CRUD Testing

TC	Test Case	Operation	Expected Output	Result
TC07	Create quiz result	Write to patients/{id}/quizResults as parent	Document created, auto-ID assigned	Pass
TC08	Read patient record	Read patients/{id} as assigned therapist	Patient data returned	Pass
TC09	Write CAP score	Write capScore=5, sirScore=3 to assessments	Document created with validated fields	Pass
TC10	Real-time sync	Therapist writes assessment; check parent app	Parent app receives update within 2 seconds	Pass

Table 12: Firestore Operations Test Cases and Results

6.3. Security Rules Testing

TC	Test Case	Attempted Action	Expected Result	Result
TC13	Parent cross-patient access	Parent A tries to read Patient B's record	Request denied	Pass
TC14	Therapist cross-patient	Therapist reads unassigned patient	Request denied	Pass
TC15	Invalid CAP score	Therapist writes capScore=9 (out of range)	Write rejected by Security Rules	Pass
TC16	Invalid SIR score	Therapist writes sirScore=-1 (out of range)	Write rejected by Security Rules	Pass
TC17	Unauthenticated read	Request without auth token to patients	Request denied	Pass

Table 13: Security and Validation Test Cases and Results

All 19 test cases passed. The backend behaves correctly under both normal usage and adversarial conditions.

7. Limitations

The following limitations apply to the current backend implementation:

1. Admin SDK dependency for role assignment

Custom claims can only be set via the Firebase Admin SDK, which runs in a trusted server environment. In the current implementation, a one-time Node.js script is run manually to set roles. This is not scalable for a production system where new accounts are created frequently. A Cloud Function triggered on new user creation would be the appropriate long-term solution.

2. Firestore Security Rules complexity

The current rules use `get()` calls to fetch parent documents for access checks (e.g., checking if a patient's `parentUid` matches the requesting user). These `get()` calls count against Firestore read quotas and can add latency to rule evaluations. For very high-frequency operations, denormalized fields in subcollection documents would be more efficient.

3. No server-side business logic

All business logic (e.g., unlock condition for the next video stage) is implemented on the client side (Android app). This means a technically sophisticated user could potentially manipulate the unlock logic by modifying client code. Moving unlock logic to a Firebase Cloud Function would enforce it server-side.

4. Firebase Spark plan read/write limits

The free-tier Firestore limits (50,000 reads/day, 20,000 writes/day) are sufficient for development and demonstration but would be exceeded in a production deployment serving hundreds of patients. Upgrading to the Blaze plan with per-use billing would be required for production scale.

5. No push notifications

The current backend does not implement Firebase Cloud Messaging (FCM) for push notifications. Reminders for therapy sessions or alerts for new assessment scores are not delivered proactively.

6. Video storage not included

Therapy videos are referenced by URL in Firestore but are not stored in Firebase Storage in the current implementation. Video hosting is outside the scope of the backend role and relies on an external CDN URL.

7. Lack of data migration tools

There is no established process for migrating data between environments (development, staging, production). This makes it difficult to move patient data or configuration changes between environments without manual intervention.

8. No audit logging

There is no built-in mechanism to track who accessed or modified data in the database. This makes it difficult to monitor for unauthorized access or to investigate data integrity issues.

9. Limited error handling in client applications

The client applications do not have robust error handling for Firestore operations. This can lead to poor user experiences when network issues or permission errors occur.

10. No data backup strategy

There is no automated process for backing up Firestore data. In case of accidental data deletion or corruption, recovery would be difficult without a backup.

11. Inefficient data structure for reporting

The current data structure may not be optimized for generating reports on patient progress or therapy outcomes. This could lead to performance issues when running complex queries.

12. No support for data versioning

There is no mechanism to track changes to patient data over time. This makes it difficult to analyze trends in patient progress or to revert to previous data states.

13. Limited scalability for user growth

The current architecture may not scale well as the number of users increases. The Firestore read/write limits and the manual role assignment process could become bottlenecks.

14. No support for offline functionality

The client applications do not have built-in support for offline operation. This means users may lose access to their data when they don't have an internet connection.

15. No automated testing framework

There is no automated testing framework for the backend services. This makes it difficult to ensure that changes to the system do not introduce new bugs or break existing functionality.

8. Conclusion & Future Scope

8.1. Conclusion

The backend infrastructure of Sound for Silence was successfully designed and implemented using Firebase's serverless cloud platform, chosen for its ability to handle real-time data synchronization, eliminate the need for managing dedicated servers, and provide built-in scalability and security. This implementation met all specified backend objectives: secure email/password authentication with role-based custom claims that distinguish between patients, therapists, and administrators; a well-structured Firestore data model that maps all application entities including user profiles, therapy sessions, exercise logs, progress metrics, and therapist assignments; comprehensive Security Rules that enforce fine-grained, role-based access control at the database layer; real-time data synchronization between the Android mobile app and the web-based therapist dashboard; and automated deployment of the therapist portal via Firebase Hosting with global CDN support and HTTPS enforcement.

A key technical contribution of this role was the design and validation of the Firestore Security Rules. Instead of embedding authorization logic within application code—which risks inconsistency or bypass—the entire access control policy was implemented declaratively at the database level. This ensures that even if a client application is compromised or modified, unauthorized data access remains impossible. The rules were rigorously tested against 19 distinct test cases covering authentication token validation, role extraction from custom claims, CRUD permissions for each user type, and adversarial scenarios such as attempts to read or modify data belonging to other users or roles. All test cases passed, confirming that data isolation between user roles is enforced correctly and consistently without any reliance on server-side application logic.

Choosing Firebase as a Backend-as-a-Service (BaaS) eliminated the need for provisioning, maintaining, or scaling backend servers, reducing infrastructure costs to effectively zero while still delivering production-grade reliability, automatic scaling under load, and enterprise-level security features like automatic TLS, DDoS protection, and audit logging. This architectural decision makes Sound for Silence immediately deployable in real-world clinical settings—such as cochlear implant rehabilitation centers—without requiring IT staff, server maintenance, or complex DevOps pipelines. From a personal development perspective, this project provided hands-on experience with

NoSQL data modeling for complex relational domains, serverless architecture design patterns, declarative security rule writing, JWT token-based authentication flows, and real-time database synchronization—all directly transferable skills for industry backend engineering roles focused on scalable, secure, and maintainable systems.

8.2. Future Scope

1. Cloud Functions for server-side logic: Firebase Cloud Functions (Node.js) would be used to implement server-side triggers — for example, automatically unlocking the next video stage after a quiz is passed, or sending FCM push notifications to the parent when a therapist records a new assessment.

2. Firebase Cloud Messaging (FCM): Implementing push notifications would allow therapists to send session reminders and parents to receive alerts for new clinical scores without opening the app.

3. Firebase App Check: App Check would be integrated to ensure that only the legitimate Android app and web dashboard can make requests to the Firestore backend, protecting against API abuse from unauthorized clients.

4. Automated role assignment via Cloud Functions: A Cloud Function triggered by `functions.auth.user().onCreate()` would automatically assign the correct role claim based on the user's email domain or a provisioning code, eliminating the manual Admin SDK script.

5. Firestore composite indexes for analytics: Adding composite indexes would allow more sophisticated queries — for example, fetching all assessments for a patient between two dates, ordered by CAP score — enabling richer analytics in the therapist dashboard.

6. Blaze plan upgrade for production: Moving from the Spark free plan to the Blaze pay-as-you-go plan would remove daily read/write limits, enabling deployment to real clinics serving hundreds of patients.

7. HIPAA/data compliance considerations: For production healthcare deployment in applicable jurisdictions, a Google Cloud Healthcare API FHIR store or a BAA (Business Associate Agreement) with Google would be required to ensure regulatory compliance for protected health information.

8. Firestore data modeling with subcollections: Structure patient documents with subcollections (assessments, sessions, media) to limit document size, simplify security rules, and enable paginated queries for therapist dashboards.

9. Security Rules with granular role checks: Implement Firestore Security Rules that enforce role-based access control (therapist vs parent vs child) at the document and subcollection level, including validation for allowed fields and write-only paths for clinicians.

10. Offline persistence and conflict resolution: Enable Firestore offline persistence on mobile clients with explicit conflict-resolution strategies (last-write-wins for metadata, merge for progress counters) to handle intermittent connectivity during home sessions.

11. End-to-end encryption for sensitive media: Store highly sensitive audio/video or assessment attachments encrypted client-side before upload, with keys managed per-patient or per-clinic to limit exposure even if storage is compromised.

12. Audit logging via Cloud Logging: Record immutable audit logs for clinical actions (assessment edits, role changes, data exports) using Cloud Logging or an append-only Firestore collection to meet clinical review and compliance needs.

13. Data export and interoperability endpoints: Provide secure, authenticated end-points or scheduled exports (CSV/NDJSON or FHIR) for clinicians to import clinical data into EHRs or research systems, with access controls and rate limits.

14. Rate limiting and abuse protection: Implement per-user and per-IP rate limiting at the API gateway or Cloud Functions level to prevent brute-force attempts, spammy writes, or accidental billing spikes.

15. CI/CD, tests, and staging environments: Establish a CI/CD pipeline (GitHub Actions/Cloud Build) with automated unit/integration tests, linting, and separate staging and production Firebase projects to validate Cloud Function changes, security rules, and migrations before release.

9. Bibliography

[1] Google LLC. **Firebase Authentication Documentation**. Available at: https://firebase.google.com/docs/auth. Accessed April 2026.

[2] Google LLC. **Cloud Firestore Documentation**. Available at: https://firebase.google.com/docs/firestore. Accessed April 2026.

[3] Google LLC. **Firebase Security Rules Reference**. Available at: https://firebase.google.com/docs/rules. Accessed April 2026.

[4] Google LLC. **Firebase Hosting Documentation**. Available at: https://firebase.google.com/docs/hosting. Accessed April 2026.

[5] Google LLC. **Firebase Admin SDK — Node.js**. Available at: https://firebase.google.com/docs/admin/setup. Accessed April 2026.

[6] Android Developers. **Jetpack Compose Documentation**. Available at: https://developer.android.com/jetpack/compose. Accessed April 2026.

[7] Nicklas Selvaggi, et al. **"Mobile Health Apps for Paediatric Audiology: A Systematic Review."** International Journal of Audiology, 2022.

[8] Chart.js Contributors. **Chart.js Documentation**. Available at: https://www.chartjs.org/docs. Accessed April 2026.

[9] Cochlear Limited. **AB CLIX Rehabilitation Application**. Available at: https://www.cochlear.com. Accessed 2025.

[10] Pressman, Roger S. **Software Engineering: A Practitioner's Approach**, 8th ed. McGraw-Hill Education, 2014.

APPENDIX A: CODING

This appendix contains the complete source code for the backend components implemented as part of the Backend & Firebase Engineer role.

9.1. A.1 Firebase Security Rules (Complete)

```
rules_version = '2';

service cloud.firestore {
  match /databases/{database}/documents {

    // ---------------------------------------------------------
    // CORE AUTHENTICATION SERVICES
    // ---------------------------------------------------------

    function isSignedIn() {
      return request.auth != null;
    }

    function getAuthUid() {
      return request.auth.uid;
    }

    function getAuthToken() {
      return request.auth.token;
    }

    function getUserRole() {
      return getAuthToken().role;
    }

    // ---------------------------------------------------------
    // ROLE-BASED ACCESS CONTROL (RBAC)
    // ---------------------------------------------------------

    function verifyRole(requiredRole) {
```

```
  return isSignedIn() && getUserRole() == requiredRole;
}

function isUserParent() {
  return verifyRole("parent");
}

function isUserTherapist() {
  return verifyRole("therapist");
}

// ---------------------------------------------------------
// DATA RETRIEVAL & RELATIONSHIP HELPERS
// ---------------------------------------------------------

function fetchPatientDocument(id) {
  return get(/databases/$(database)/documents/patients/$(id)).data;
}

function isParentOfPatient(id) {
  return fetchPatientDocument(id).parentUid == getAuthUid();
}

function isTherapistOfPatient(id) {
  return fetchPatientDocument(id).therapistUid == getAuthUid();
}

// ---------------------------------------------------------
// FIELD-LEVEL VALIDATION LOGIC
// ---------------------------------------------------------

function isIntegerInRange(value, min, max) {
  return value is int &&
         value >= min &&
         value <= max;
}
```

```
function validateCapScore(data) {
  return data.hasAll(['capScore']) &&
         isIntegerInRange(data.capScore, 0, 7);
}

function validateSirScore(data) {
  return data.hasAll(['sirScore']) &&
         isIntegerInRange(data.sirScore, 0, 5);
}

function validateAssessmentSchema() {
  let incomingData = request.resource.data;
  return validateCapScore(incomingData) &&
         validateSirScore(incomingData);
}

// ---------------------------------------------------------
// SECURITY RULES DEFINITIONS
// ---------------------------------------------------------

// User Profile Management
match /users/{uid} {
  function isSelf(id) {
    return getAuthUid() == id;
  }

  allow read: if isSignedIn() && isSelf(uid);
  allow write: if isSignedIn() && isSelf(uid);
}

// Patient and Clinical Data
match /patients/{patientId} {

  function canReadPatientBase() {
    return (isUserParent() && isParentOfPatient(patientId)) ||
```

```
            (isUserTherapist() && isTherapistOfPatient(patientId));
    }

    function canWritePatientBase() {
      return isUserTherapist() && isTherapistOfPatient(patientId);
    }

    allow read: if canReadPatientBase();
    allow write: if canWritePatientBase();

    // Nested: Quiz Results
    match /quizResults/{resultId} {
      function canManageQuiz() {
        return isUserParent() && isParentOfPatient(patientId);
      }

      function canViewQuiz() {
        return isUserTherapist() && isTherapistOfPatient(patientId);
      }

      allow read: if canManageQuiz() || canViewQuiz();
      allow write: if canManageQuiz();
    }

    // Nested: Clinical Assessments
    match /assessments/{assessmentId} {
      function canViewAssessment() {
        return (isUserTherapist() && isTherapistOfPatient(patientId))
||
              (isUserParent() && isParentOfPatient(patientId));
      }

      function canSubmitAssessment() {
        return isUserTherapist() &&
              isTherapistOfPatient(patientId) &&
              validateAssessmentSchema();
```

```
        }

        allow read: if canViewAssessment();
        allow create: if canSubmitAssessment();
        allow update: if canSubmitAssessment();
        allow delete: if false;
      }
    }

    // Static Content and Curriculum
    match /stages/{stageId} {
      function isSystemRead() {
        return isSignedIn();
      }

      allow read: if isSystemRead();
      allow write: if false;

      match /videos/{videoId} {
        allow read: if isSystemRead();
        allow write: if false;
      }
    }

    // Root-Level Lockdown
    match /{allPaths=**} {
      allow read, write: if false;
    }
  }
}
```

9.2. A.2 Admin SDK — Role Assignment Script (set-role.js)

```
const admin = require("firebase-admin");
const serviceAccount = require("./serviceAccountKey.json");

// Initialize Firebase Admin SDK
admin.initializeApp({
```

```
  credential: admin.credential.cert(serviceAccount),
});

const VALID_ROLES = ["parent", "therapist", "admin", "user"];

/**
 * Validates if the provided role is allowed
 * @param {string} role - The role to validate
 * @returns {boolean} - True if role is valid
 */
function isValidRole(role) {
  return VALID_ROLES.includes(role.toLowerCase());
}

/**
 * Sets custom claims (role) for a user in Firebase Authentication
 * @param {string} uid - The user's unique identifier
 * @param {string} role - The role to assign to the user
 * @returns {Promise<void>}
 */
async function setUserRole(uid, role) {
  try {
    // Validate inputs
    if (!uid || typeof uid !== "string") {
      throw new Error("Invalid UID: must be a non-empty string");
    }

    if (!role || typeof role !== "string") {
      throw new Error("Invalid role: must be a non-empty string");
    }

    const normalizedRole = role.toLowerCase();

    if (!isValidRole(normalizedRole)) {
      throw new Error(
        `Invalid role '${role}'. Allowed roles: ${VALID_ROLES.join(",
```

```
")}`
      );
    }

    // Set custom claims for the user
    await admin.auth().setCustomUserClaims(uid, { role:
normalizedRole });
    console.log(`✓ SUCCESS: Role '${normalizedRole}' assigned to UID:
${uid}`);
  } catch (error) {
    console.error(`✗ ERROR: Failed to set role for UID ${uid}`);
    console.error(`  Details: ${error.message}`);
    process.exit(1);
  }
}

/**
 * Retrieves and displays the current role for a user
 * @param {string} uid - The user's unique identifier
 * @returns {Promise<void>}
 */
async function getUserRole(uid) {
  try {
    const user = await admin.auth().getUser(uid);
    const role = user.customClaims?.role || "none";
    console.log(`User UID: ${uid}`);
    console.log(`Current role: ${role}`);
  } catch (error) {
    console.error(`ERROR: Could not retrieve user data for UID ${uid}
`);
    console.error(`  Details: ${error.message}`);
  }
}

/**
 * Removes all custom claims from a user
```

```
 * @param {string} uid - The user's unique identifier
 * @returns {Promise<void>}
 */
async function removeUserRole(uid) {
  try {
    await admin.auth().setCustomUserClaims(uid, null);
    console.log(`✓ SUCCESS: All roles removed for UID: ${uid}`);
  } catch (error) {
    console.error(`ERROR: Failed to remove roles for UID ${uid}`);
    console.error(`  Details: ${error.message}`);
    process.exit(1);
  }
}

/**
 * Displays usage instructions
 */
function showUsage() {
  console.log("Firebase Role Management Tool");
  console.log("==============================");
  console.log("");
  console.log("Usage:");
  console.log("  Set role:    node set-role.js set <uid> <role>");
  console.log("  Get role:    node set-role.js get <uid>");
  console.log("  Remove role: node set-role.js remove <uid>");
  console.log("");
  console.log(`Valid roles: ${VALID_ROLES.join(", ")}`);
  console.log("");
  console.log("Examples:");
  console.log("  node set-role.js set user123 parent");
  console.log("  node set-role.js get user123");
  console.log("  node set-role.js remove user123");
}

// Main execution
async function main() {
```

```
const args = process.argv.slice(2);

if (args.length === 0) {
  showUsage();
  process.exit(0);
}

const command = args.toLowerCase();

switch (command) {
  case "set":
    if (args.length !== 3) {
      console.error("Error: set command requires <uid> and <role>");
      showUsage();
      process.exit(1);
    }
    await setUserRole(args, args);
    break;

  case "get":
    if (args.length !== 2) {
      console.error("Error: get command requires <uid>");
      showUsage();
      process.exit(1);
    }
    await getUserRole(args);
    break;

  case "remove":
    if (args.length !== 2) {
      console.error("Error: remove command requires <uid>");
      showUsage();
      process.exit(1);
    }
    await removeUserRole(args);
    break;
```

```
    default:
      console.error(`Unknown command: '${command}'`);
      showUsage();
      process.exit(1);
  }
}

main().catch((error) => {
  console.error("Fatal error:", error.message);
  process.exit(1);
});
```

9.3. A.3 Admin SDK — Firestore Seed Script (seed.js)

```
const admin = require("firebase-admin");
const serviceAccount = require("./serviceAccountKey.json");

admin.initializeApp({
  credential: admin.credential.cert(serviceAccount),
});

const db = admin.firestore();

const stagesData = [
  {
    id: "stage_01",
    data: {
      title: "Stage 1: Sound Detection",
      order: 1,
      description:
        "Learning to detect the presence and absence of sound.",
    },
    videos: [
      {
        id: "video_01_01",
        data: {
          title: "What is Sound?",
```

```
          videoUrl: "https://cdn.example.com/videos/s1v1.mp4",
          order: 1,
          quizQuestions: [
            {
              question: "What do we call vibrations that we can hear?",
              options: ["Light", "Sound", "Heat", "Touch"],
              correctIndex: 1,
            },
          ],
        },
      },
    ],
  },
  {
    id: "stage_02",
    data: {
      title: "Stage 2: Sound Discrimination",
      order: 2,
      description:
        "Distinguishing between different environmental sounds.",
    },
    videos: [],
  },
];

async function seedData() {
  for (const stage of stagesData) {
    await db.collection("stages").doc(stage.id).set(stage.data);
    console.log(`Seeded stage: ${stage.id}`);

    for (const video of stage.videos) {
      await db
        .collection("stages")
        .doc(stage.id)
        .collection("videos")
        .doc(video.id)
```

```
        .set(video.data);
      console.log(`  Seeded video: ${video.id}`);
    }
  }
  console.log("Seeding complete.");
}

seedData().catch(console.error);
```

9.4. A.4 Firebase Hosting Configuration (firebase.json)

```
{
  "hosting": {
    "public": "public",
    "ignore": ["firebase.json", "**/.*", "**/node_modules/**"],
    "rewrites": [
      {
        "source": "**",
        "destination": "/index.html"
      }
    ],
    "headers": [
      {
        "source": "**",
        "headers": [
          {
            "key": "Cache-Control",
            "value": "no-cache"
          }
        ]
      }
    ]
  }
}
```

9.5. A.5 Kotlin — Firebase Auth with Role Redirect

```
// LoginViewModel.kt
import androidx.lifecycle.ViewModel
```

```
import androidx.lifecycle.viewModelScope
import com.google.firebase.auth.FirebaseAuth
import kotlinx.coroutines.tasks.await
import kotlinx.coroutines.launch

/**
 * LoginViewModel
 *
 * - Uses FirebaseAuth for email/password sign-in.
 * - Uses Kotlin coroutines (viewModelScope + Tasks.await) for concise
async handling.
 * - Returns role on success via callbacks and provides more robust
error handling.
 * - Includes optional forceRefresh parameter to fetch a fresh ID token
when needed.
 */
class LoginViewModel : ViewModel() {

  private val auth: FirebaseAuth = FirebaseAuth.getInstance()

  /**
   * Attempts to sign in with email/password and fetch the user's role
custom claim.
   *
   * @param email user email
   * @param password user password
   * @param forceRefresh if true requests a fresh token from the server
(useful when claims may have changed)
   * @param onSuccess called with the role string on success
   * @param onError called with a user-facing error message on failure
   */
  fun login(
    email: String,
    password: String,
    forceRefresh: Boolean = false,
    onSuccess: (String) -> Unit,
```

```
    onError: (String) -> Unit
  ) {
    viewModelScope.launch {
      try {
        // Sign in
        auth.signInWithEmailAndPassword(email, password).await()

        // Ensure currentUser is non-null
        val user = auth.currentUser
        if (user == null) {
          onError("Authentication succeeded but no user is available.")
          return@launch
        }

        // Get ID token; forceRefresh can be true to ensure latest
custom claims
        val tokenResult = user.getIdToken(forceRefresh).await()
        val claims = tokenResult.claims

        // Safely extract the 'role' claim
        val role = when (val r = claims["role"]) {
          is String -> r
          is CharSequence -> r.toString()
          else -> "unknown"
        }

        onSuccess(role)
      } catch (e: Exception) {
        // Map common Firebase exceptions to friendlier messages
        val message = when (e) {
          is com.google.firebase.auth.FirebaseAuthInvalidUserException
-> "No account found for that email."
          is
com.google.firebase.auth.FirebaseAuthInvalidCredentialsException ->
"Invalid credentials. Check email and password."
          else -> e.message ?: "Login failed. Please try again."
```

```
        }
        onError(message)
      }
    }
  }

  /**
   * Optional helper to sign out the current user.
   */
  fun logout() {
    auth.signOut()
  }

  /**
   * Optional helper to get the currently cached role synchronously if
available.
   * Returns null if no signed-in user or no cached token present.
   */
  fun getCachedRole(): String? {
    val user = auth.currentUser ?: return null
    val tokenResult = runCatching
{ user.getIdToken(false).result }.getOrNull() ?: return null
    val claim = tokenResult.claims["role"]
    return when (claim) {
      is String -> claim
      is CharSequence -> claim.toString()
      else -> null
    }
  }
}
```

9.6. A.6 JavaScript — Real-Time Assessments Listener

```
// assessments.js — therapist dashboard (expanded with error handling,
filtering, pagination, and unsubscribe safety)
import {
  collection,
  query,
```

```
  where,
  orderBy,
  limit,
  startAfter,
  onSnapshot,
  getDocs,
  getCountFromServer,
} from "firebase/firestore";
import { db, auth } from "./firebase-config.js";

/**
 * subscribeToAssessments
 *
 * Real-time subscription to a patient's assessments with:
 * - ordering by assessmentDate (ascending)
 * - optional status filtering
 * - optional pagination (pageSize + startAfterDoc)
 * - robust error handling and unsubscribe safety
 *
 * @param {string} patientId - Firestore patient document id
 * @param {{status?: string, pageSize?: number, startAfterDoc?:
QueryDocumentSnapshot|null}} options
 * @param {(assessments: Array, meta: {hasMore: boolean, totalCount?:
number}) => void} callback
 * @returns {function():void} unsubscribe function
 */
export function subscribeToAssessments(
  patientId,
  options = {},
  callback
) {
  const { status = null, pageSize = 25, startAfterDoc = null } =
options;

  if (!patientId) {
    throw new Error("patientId is required for
```

```
subscribeToAssessments");
  }

  // Build base query
  let q = query(
    collection(db, "patients", patientId, "assessments"),
    orderBy("assessmentDate", "asc")
  );

  // Apply status filter if provided
  if (status) {
    q = query(q, where("status", "==", status));
  }

  // Apply pagination limit and cursor
  if (pageSize) {
    q = query(q, limit(pageSize));
  }
  if (startAfterDoc) {
    q = query(q, startAfter(startAfterDoc));
  }

  // Optional: fetch total count for UI (not real-time; separate one-
time request)
  let totalCount;
  getCountFromServer(collection(db, "patients", patientId,
"assessments"))
    .then((snapshot) => {
      totalCount = snapshot.data().count;
    })
    .catch(() => {
      totalCount = undefined; // ignore count errors
    });

  // Attach real-time listener
  const unsubscribe = onSnapshot(
```

```
    q,
    (snapshot) => {
      const assessments = snapshot.docs.map((doc) => ({
        id: doc.id,
        ...doc.data(),
      }));

      // Determine if there are more pages (naive check: page full ->
maybe more)
      const hasMore = snapshot.docs.length === pageSize;

      callback(assessments, { hasMore, totalCount });
    },
    (error) => {
      // Pass errors to callback as empty array + meta.error for
consumer to surface
      console.error("subscribeToAssessments error:", error);
      callback([], { hasMore: false, totalCount, error });
    }
  );

  // Return unsubscribe function for caller to detach listener
  return unsubscribe;
}

// Example usage in dashboard.js
// const unsubscribe = subscribeToAssessments(currentPatientId,
{ status: "completed", pageSize: 50 }, (data, meta) => {
//   if (meta.error) {
//     showToast("Failed to load assessments");
//     return;
//   }
//   renderProgressChart(data);
//   if (meta.hasMore) {
//     showLoadMoreButton();
//   }
```

```
// });
//
// // Call unsubscribe() when leaving the page to detach listener
// // unsubscribe();
```

APPENDIX B: ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
APK	Android Package Kit
Auth	Authentication
BaaS	Backend as a Service
BAA	Business Associate Agreement
BOM	Bill of Materials
CAP	Categories of Auditory Performance
CDN	Content Delivery Network
CI	Cochlear Implant
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DFD	Data Flow Diagram
ER	Entity-Relationship
FCM	Firebase Cloud Messaging
FHIR	Fast Healthcare Interoperability Resources
HIPAA	Health Insurance Portability and Accountability Act
HOD	Head of Department
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HTTP Secure
IDE	Integrated Development Environment
JS	JavaScript
JSON	JavaScript Object Notation
VPS	Virtual Private Server

Table 14: Abbreviations and Acronyms